

DataLab Interseguro

Documentacion Tecnica: Arquitectura, Queries y Machine Learning

Proyecto datalab-interseguro | insurance_claims | Google Cloud Platform

1. Arquitectura de Datos — Patron Medallion en GCP

El pipeline completo sigue el patron Medallion (Bronze → Silver → Gold), un estandar de la industria para data lakes modernos. Cada capa tiene responsabilidad unica y el dato se enriquece progresivamente hasta quedar listo para ML.

1.1 Stack tecnologico

Componente	Rol en el proyecto
Google Cloud Storage (GCS)	Landing zone del CSV raw de Kaggle. Bucket: gs://datalab-interseguro-2026
BigQuery	Data warehouse para las 3 capas Medallion. Dataset: insurance_claims
BigQuery ML (BQML)	Entrenamiento, evaluacion y prediccion del modelo directamente en SQL
SQL Standard (BQ)	100% de la logica: transformaciones, feature engineering, ML

1.2 Flujo del pipeline

```
GCS Bucket (CSV raw, 180k filas)
|
v LOAD DATA OVERWRITE
bronze_eventos      <- Capa cruda, sin transformacion
|
v CREATE TABLE AS SELECT (limpieza + tipado + particion)
silver_eventos      <- Capa limpia, particionada por fecha
|
v CREATE TABLE AS SELECT (aggregations por case_id)
gold_features        <- 1 fila por caso, 20 features + label
|
v CREATE MODEL (BQML BOOSTED_TREE_CLASSIFIER)
modelo_anomalias    <- Modelo entrenado en BigQuery
|
```

```
v ML.EVALUATE -&gt; metricas de calidad
v ML.PREDICT -&gt; predicciones accionables
```

1.3 Descripcion de cada capa

Capa	Tabla	Descripcion	Filas
Bronze	bronze_eventos	Datos crudos cargados desde GCS. Preserva integridad del original. Schema auto-detectado.	120,800
Silver	silver_eventos	Datos limpios, tipados y particionados por fecha del primer evento. Datos eliminados.	180,000*
Gold	gold_features	Feature store: 1 fila por case_id con 20 variables derivadas del comportamiento procesal.	30,000
ML	modelo_anomalias	BOOSTED_TREE_CLASSIFIER entrenado con BQML. Label: model_es_happy_path.	Modelo

* Particionada por fecha para optimizar consultas temporales.

2. Queries SQL — Implementacion Completa del Pipeline

2.1 Carga Bronze — Ingesta desde GCS

El primer paso es cargar el CSV desde el bucket de GCS directamente a BigQuery usando LOAD DATA. Esta operacion crea la tabla bronze sin ninguna transformacion, preservando el dato original exactamente como viene de la fuente.

```
LOAD DATA OVERWRITE `datalab-interseguro.insurance_claims.bronze_eventos`  
FROM FILES (  
  format = 'CSV',  
  uris = ['gs://datalab-interseguro-2026/Insurance_claims_event_log.csv'],  
  skip_leading_rows = 1,  
  autodetect = TRUE  
);
```

2.2 Creacion Silver — Limpieza y Particionado

La capa Silver aplica transformaciones de calidad: cast de tipos, eliminacion de nulos criticos y particionado por fecha para optimizar el costo de queries futuras. La particion es clave en BigQuery porque reduce el volumen de datos escaneados.

```
CREATE OR REPLACE TABLE `datalab-interseguro.insurance_claims.silver_eventos`  
PARTITION BY DATE(timestamp_evento)  
AS  
SELECT  
  CAST(case_id AS STRING) AS case_id,  
  CAST(activity AS STRING) AS actividad,  
  CAST(timestamp AS TIMESTAMP) AS timestamp_evento,  
  CAST(resource AS STRING) AS recurso,  
  CAST(claim_amount AS FLOAT64) AS claim_amount,  
  CAST(claimant_age AS INT64) AS claimant_age,  
  CAST(type_of_policy AS STRING) AS type_of_policy,  
  CAST(car_make AS STRING) AS car_make,  
  CAST(car_year AS INT64) AS car_year,  
  CAST(type_of_accident AS STRING) AS type_of_accident  
FROM `datalab-interseguro.insurance_claims.bronze_eventos`  
WHERE case_id IS NOT NULL  
  AND timestamp IS NOT NULL;
```

2.3 Creacion Gold — Feature Engineering

Este es el query mas importante del pipeline. Agrupa los eventos por case_id y deriva 20 variables que capturan el comportamiento procesal de cada siniestro. Aqui se construye tambien el label es_happy_path que el modelo aprendera a predecir.

```
CREATE OR REPLACE TABLE `datalab-interseguro.insurance_claims.gold_features`
AS
WITH eventos_por_caso AS (
  SELECT
    case_id,
    -- Atributos del caso (invariantes, tomar el primer valor)
    MAX(claim_amount)      AS claim_amount,
    MAX(claimant_age)      AS claimant_age,
    MAX(type_of_policy)    AS type_of_policy,
    MAX(car_make)          AS car_make,
    MAX(car_year)          AS car_year,
    MAX(type_of_accident)  AS type_of_accident,

    -- Features procesales derivadas
    COUNT(*)                AS num_eventos_rpa,
    MIN(timestamp_evento)   AS ts_inicio,
    MAX(timestamp_evento)   AS ts_fin,
    DATE_DIFF(DATE(MAX(timestamp_evento)),
              DATE(MIN(timestamp_evento)), DAY) AS duracion_total_dias,

    -- Posicion de eventos clave en la secuencia del caso
    MIN(CASE WHEN actividad = 'FNOL'      THEN rn END) AS posicion_fnol,
    MIN(CASE WHEN actividad = 'Payment' THEN rn END) AS posicion_pago,
    MIN(CASE WHEN actividad = 'Decision' THEN rn END) AS posicion_decision,

    -- Duraciones entre hitos
    DATE_DIFF(
      DATE(MIN(CASE WHEN actividad='Payment' THEN timestamp_evento END)),
      DATE(MIN(CASE WHEN actividad='FNOL'      THEN timestamp_evento END)),
      DAY) AS dias_fnol_a_pago,
    DATE_DIFF(
      DATE(MIN(CASE WHEN actividad='Payment' THEN timestamp_evento END)),
      DATE(MIN(CASE WHEN actividad='Decision' THEN timestamp_evento END)),
      DAY) AS dias_decision_a_pago,

    -- Flags binarios de orden procesal
    CASE WHEN
      MIN(CASE WHEN actividad='FNOL'      THEN rn END) <
      MIN(CASE WHEN actividad='Payment' THEN rn END)
    THEN 1 ELSE 0 END AS fnol_antes_pago,
    CASE WHEN
      COUNTIF(actividad LIKE '%RPA%') > 0
    THEN 1 ELSE 0 END AS tiene_rpa
  FROM (
    SELECT *,
      ROW_NUMBER() OVER (PARTITION BY case_id
                        ORDER BY timestamp_evento) AS rn
    FROM `datalab-interseguro.insurance_claims.silver_eventos`
  )
  GROUP BY case_id
)
```

```
SELECT
  *,
  -- LABEL: construido desde el comportamiento procesal
  -- Un caso es happy_path si sigue el orden esperado y tiene duracion normal
  CASE WHEN
    fnol_antes_pago = 1
    AND posicion_fnol IS NOT NULL
    AND posicion_pago IS NOT NULL
    AND duracion_total_dias BETWEEN 15 AND 60
    AND dias_fnol_a_pago >= 0
  THEN 1 ELSE 0
  END AS es_happy_path
FROM eventos_por_caso;
```

Nota: es_happy_path es un label construido con reglas de negocio basadas en Process Mining, no una etiqueta de fraude explícita del CSV original. El modelo aprende a generalizar mas alla de estas reglas.

3. Queries de Machine Learning con BigQuery ML

3.1 Entrenamiento del Modelo

BigQuery ML permite entrenar modelos directamente desde SQL sin mover datos fuera de BigQuery. El CREATE MODEL lee gold_features, reserva automaticamente un split de validacion, entrena el Gradient Boosting y guarda el modelo en el dataset.

```
CREATE OR REPLACE MODEL
  `datalab-interseguro.insurance_claims.modelo_anomalias`
OPTIONS(
  model_type          = 'BOOSTED_TREE_CLASSIFIER',
  input_label_cols    = ['es_happy_path'],
  data_split_method   = 'AUTO_SPLIT',
  early_stop          = TRUE,
  min_rel_progress     = 0.01,
  enable_global_explain = TRUE    -- permite SHAP values para interpretabilidad
) AS
SELECT
  -- Features (14 variables)
  claim_amount,
  claimant_age,
  type_of_policy,
  car_make,
  car_year,
  type_of_accident,
  num_eventos_rpa,
  posicion_fnol,
  posicion_pago,
  duracion_total_dias,
  dias_fnol_a_pago,
  dias_decision_a_pago,
  fnol_antes_pago,
  tiene_rpa,
  -- Label
  es_happy_path
FROM `datalab-interseguro.insurance_claims.gold_features`
WHERE es_happy_path IS NOT NULL;
```

3.2 Evaluacion del Modelo

```
SELECT
  ROUND(precision, 4) AS precision,
  ROUND(recall, 4) AS recall,
```

```

ROUND(accuracy, 4) AS accuracy,
ROUND(f1_score, 4) AS f1_score,
ROUND(log_loss, 4) AS log_loss,
ROUND(roc_auc, 4) AS roc_auc
FROM
  ML.EVALUATE(
    MODEL `datalab-interseguro.insurance_claims.modelo_anomalias`,
    (
      SELECT * FROM
        `datalab-interseguro.insurance_claims.gold_features`
        WHERE es_happy_path IS NOT NULL
    )
  );

-- Resultado:
-- precision: 0.9941 recall: 0.9001 accuracy: 0.8973
-- f1_score: 0.9448 log_loss: 0.2510 roc_auc: 0.9215

```

3.3 Predicciones sobre nuevos casos

```

SELECT
  p.case_id,
  g.claim_amount,
  g.type_of_accident,
  g.duracion_total_dias,
  p.predicted_es_happy_path,
  -- Extraer la probabilidad de ser anomalo (clase 0)
  ROUND(
    (SELECT prob.prob FROM UNNEST(predicted_es_happy_path_probs) AS prob
     WHERE prob.label = 0),
    4) AS probabilidad_anomalia
FROM
  ML.PREDICT(
    MODEL `datalab-interseguro.insurance_claims.modelo_anomalias`,
    (
      SELECT * FROM
        `datalab-interseguro.insurance_claims.gold_features`
    )
  ) AS p
JOIN `datalab-interseguro.insurance_claims.gold_features` AS g
  ON p.case_id = g.case_id
WHERE p.predicted_es_happy_path = 0          -- solo los anomalos
ORDER BY probabilidad_anomalia DESC
LIMIT 20;

```

4. Justificacion Tecnica del Modelo de ML

4.1 Por que BOOSTED_TREE_CLASSIFIER?

La eleccion del algoritmo no fue arbitraria. Gradient Boosting (el algoritmo detras de BOOSTED_TREE_CLASSIFIER) es el estandar de la industria para deteccion de fraude con datos tabulares estructurados. Estas son las razones tecnicas:

Criterio	Por que Gradient Boosting gana
Datos mixtos (numericos + categoricos)	Maneja nativamente STRING (type_of_accident, car_make) sin necesidad de one-hot encoding manual
Dataset desbalanceado (2.45% anomalias)	Los arboles de decision son robustos al desbalance de clases. No requiere SMOTE ni undersampling
Features correlacionadas	duracion_total_dias y dias_finol_a_pago estan correlacionadas. Los arboles manejan multicolinealidad
Interpretabilidad	BQML soporta ML.EXPLAIN_PREDICT con SHAP values, permitiendo explicar por que cada caso fue
Precision critica	Gradient Boosting tiende a maximizar precision en datasets con pocas muestras positivas, lo cual es ic
No requiere normalizacion	claim_amount (miles de dolares) y posicion_finol (numeros pequenos) no necesitan escalado. Los arbo

4.2 Como funciona BOOSTED_TREE_CLASSIFIER internamente

Gradient Boosting construye un ensemble de arboles de decision de forma secuencial. Cada arbol nuevo corrige los errores del anterior:

```
Iteracion 1: Arbol_1 aprende patrones basicos del dataset
             Residuo_1 = Label_real - Prediccion_Arbol_1

Iteracion 2: Arbol_2 aprende a predecir Residuo_1
             Residuo_2 = Residuo_1 - Prediccion_Arbol_2

Iteracion N: Arbol_N aprende a predecir Residuo_(N-1)

Prediccion final = suma ponderada de todos los arboles:
P(anomalia) = Arbol_1(x) + lr*Arbol_2(x) + lr*Arbol_3(x) + ...
donde lr = learning_rate (penaliza cada arbol para evitar overfitting)

Con early_stop=TRUE: el entrenamiento se detiene cuando
improvement < min_rel_progress (0.01) en el validation set
-> Evita overfitting automaticamente sin tunear n_estimators
```

4.3 Como se construyo el label es_happy_path

Este es un punto critico: no habia una columna 'es_fraude' en el dataset de Kaggle. El label fue construido aplicando reglas de Process Mining sobre el event log:

Condicion del label	Justificacion de negocio
fnol_antes_pago = 1	El FNOL (First Notice of Loss) DEBE ocurrir antes del pago. Si el pago se registra antes del a
posicion_fnol IS NOT NULL	El caso debe tener un evento FNOL. Sin aviso de perdida no hay proceso valido.
posicion_pago IS NOT NULL	El caso debe haber llegado a pago. Sin pago no puede evaluarse el ciclo completo.
duracion_total_dias BETWEEN 15 AND 60	Menos de 15 dias: demasiado rapido para ser legítimo. Mas de 60: abandonado o congelado
dias_fnol_a_pago >= 0	El pago no puede ocurrir antes del FNOL. Si el valor es negativo, hay manipulacion de timest

Un caso es es_happy_path=1 si cumple TODAS las condiciones. Cualquier violacion produce es_happy_path=0 (anomalo). El resultado: 97.55% happy path / 2.45% anomalos.

4.4 Por que el modelo aprende mas que las reglas

Una pregunta valida es: si ya tenemos reglas para construir el label, para que necesitamos ML? La respuesta es que el modelo generaliza:

- Las reglas detectan violaciones EXPLICITAS (FNOL despues de pago, duracion fuera de rango)
- El modelo detecta COMBINACIONES sutiles: un caso que pasa todas las reglas individualmente pero cuya COMBINACION de features es estadisticamente inusual
- Ejemplo: claim_amount alto + duracion corta + tipo Head-on es una combinacion que el modelo aprende a desconfiar aunque ninguna variable por separado viole una regla
- En produccion, el fraude evolucionara para evitar las reglas conocidas — el modelo se readapta con reentrenamiento

4.5 Interpretacion de las metricas

99.4%	90.0%	94.5%	92.2%
Precision	Recall	F1-Score	ROC-AUC

Metrica	Formula simplificada	Resultado	Implicacion para el negocio
Precision	$VP / (VP + FP)$	99.4%	De cada 100 alertas generadas, 99.4 son casos realmente anomalos. Costo
Recall	$VP / (VP + FN)$	90.0%	El modelo detecta 9 de cada 10 fraudes reales. El 10% restante escapa —
F1-Score	$2*(P*R)/(P+R)$	94.5%	Balance optimo. Un F1 > 90% con desbalance 97.5/2.5 es calidad de produ
ROC-AUC	Area bajo curva ROC	92.2%	El modelo separa las clases correctamente el 92.2% de las veces. AUC > 0
Log Loss	$-\log(P(\text{clase correcta}))$	0.251	Las probabilidades predichas estan bien calibradas. Util para ajustar el umbr

4.6 Como ajustar el umbral de decision

BQML por default clasifica como anomalo cuando $P(0) > 0.5$. En produccion, Interseguro puede ajustar esto segun tolerancia al riesgo:

```
-- Umbral conservador (prioriza Recall: capturar mas fraudes, acepta mas FP)
WHERE probabilidad_anomalia > 0.30  -- detecta mas casos, algunos seran falsos

-- Umbral actual por defecto
WHERE probabilidad_anomalia > 0.50  -- balance precision/recall

-- Umbral estricto (prioriza Precision: solo casos de alta certeza)
WHERE probabilidad_anomalia > 0.90  -- como los 20 casos del demo (0.9698)

-- Regla practica para Interseguro:
-- P > 0.95 -> investigacion automatica obligatoria
-- P 0.70-0.95 -> revision humana prioritaria
-- P 0.50-0.70 -> monitoreo pasivo
```

5. Importancia de Features y Proximo Paso: SHAP

5.1 Features mas importantes (estimacion basada en logica del modelo)

BQML con `enable_global_explain=TRUE` permite consultar `ML.EXPLAIN_MODEL` para obtener la importancia real de cada feature. Basado en la logica del modelo y los hallazgos del EDA, el ranking estimado es:

Ranking	Feature	Por que es importante
1	<code>fnol_antes_pago</code>	Flag binario critico: violacion directa del orden procesal. Diferencia clara entre clases.
2	<code>duracion_total_dias</code>	Variable continua con separacion clara: 29 dias (anomalo) vs 34 dias (normal).
3	<code>dias_fnol_a_pago</code>	Velocidad del proceso: anomalos cierran mas rapido en la ventana FNOL-Pago.
4	<code>claim_amount</code>	Los anomalos tienen monto levemente mayor (\$5,692 vs \$5,493) — senal debil pero consistente.
5	<code>posicion_fnol / posicion_pago</code>	El orden relativo de los hitos en la secuencia de eventos del caso.
6	<code>type_of_accident</code>	Head-on y Rollover aparecen mas frecuentemente en casos anomalos.
7	<code>num_eventos_rpa</code>	Casos con muchas actividades repetidas pueden indicar reprocesamiento.

5.2 Query para obtener importancia real (SHAP)

```
-- Importancia global de features (ejecutar post-entrenamiento)
SELECT
  feature,
  ROUND(attribution, 4) AS importancia_shap
FROM
  ML.EXPLAIN_MODEL(
    MODEL `datalab-interseguro.insurance_claims.modelo_anomalias`
  )
ORDER BY ABS(attribution) DESC;

-- Explicacion a nivel de caso individual (por que este caso fue flaggeado)
SELECT
  case_id,
  feature_name,
  ROUND(attribution, 4) AS contribucion_al_score
FROM
  ML.EXPLAIN_PREDICT(
    MODEL `datalab-interseguro.insurance_claims.modelo_anomalias`,
```

```
(SELECT * FROM `datalab-interseguro.insurance_claims.gold_features`  
  WHERE case_id = '9f168519-acd4-471a-8bbb-2c6d6b2986aa')  
)  
ORDER BY ABS(attribution) DESC;
```

ML.EXPLAIN_PREDICT es el paso de interpretabilidad que transforma el modelo de 'caja negra' a una herramienta auditada — requerimiento regulatorio en la industria aseguradora.

5.3 Limitaciones conocidas y mejoras futuras

- Recall 90%: el 10% de casos reales que escapa puede reducirse ajustando el umbral a 0.30 o con `class_weight_balanced=TRUE` en el `CREATE MODEL`
- Label construido con reglas: en produccion, etiquetas validadas por el equipo de fraude mejoraran la calidad del modelo significativamente
- Dataset de Kaggle: las distribuciones pueden diferir de los datos reales de Interseguro — se recomienda reentrenamiento con datos propios
- No hay drift monitoring: en produccion se requiere `ML.EVALUATE` periodico para detectar degradacion del modelo con datos nuevos
- Cloud Functions pendiente: automatizaria la ingesta (GCS trigger) y el ciclo completo de prediccion diaria sin intervencion manual

DataLab Interseguro — Primera Edicion 2026 | En colaboracion con Google Cloud & ETH Consultores | Documento de uso interno